

Reconsider the N-Queens problem described in a previous exercise.

Define pseudo-code (and possibly an implementation in any high-level language) of a portfolio of programs that, given an instance of the problem (namely, a value for integer $N > 3$) find a placement of N queens in a $N \times N$ chessboard such that no queen is under attack.

Each program in the portfolio shall use a solving method inspired by a local search iterative improvement search state exploration algorithm shown in the course, and in particular:

- 1. steepest ascent/descent with restarts
- 2. first-choice hill climbing with restarts
- 3. simulated annealing.

Steepest ascent/descent

```
SteepestDescent(N):
    stato = generaStatoCasuale(N)

    while true:
        if h(stato) == 0:
            return stato

        vicini = generaVicini(stato)

        migliore = stato

        foreach vicino in vicini:
            if h(vicino) < h(migliore):
                migliore = vicino

        if h(migliore) < h(stato):
            stato = migliore
        else:    # restart
            stato = vicini.getRandom()
```

HillClimb

```
HillClimbing(N):
    stato = generaStatoCasuale(N)
```

```

while true:
    if h(stato) == 0:
        return stato

    vicini = generaVicini(stato)
    trovato = false

    foreach vicino in vicini:
        if h(vicino) < h(stato):
            stato = vicino
            trovato = vero
            interrompi il ciclo

    if trovato == falso:
        stato = generaStatoCasuale(N)

```

Annealing

```

SimulatedAnnealing(N):
    stato = generaStatoCasuale(N)

    for T from 0 to inf do:
        if h(stato) == 0:
            return stato

        vicino = generaVicinoCasuale(stato)
        deltaE = h(vicino) - h(stato)

        if deltaE < 0:
            stato = vicino
        else:
            prob = exp(-deltaE / T)

            if random(0,1) < p:
                stato = vicino

    return fallimento

```